```
  def self.deprecate(old_method, new_method)
    define_method(old_method) do |*args, &block|
      warn "Warning: #{old_method}() is deprecated. Use #{new_method}()."
      send(new_method, *args, &block)
    end
  end

  deprecate :GetTitle, :title
  deprecate :LEND_TO_USER, :lend_to
  deprecate :title2, :subtitle
end
```

The deprecate method takes the old name and the new name of a method and defines a *Dynamic Method (51)* that catches calls to the old name. The Dynamic Method forwards the calls to the renamed method—but first it prints a warning on the console to notify the callers that the old name has been deprecated:

```
b = Book.new
b.LEND_TO_USER("Bill")
```

```
Warning: LEND_TO_USER() is deprecated. Use lend_to().
Lending to Bill
```

That was an ingenious way to use a Class Macro. However, if you really want to understand Class Macros, as well as Singleton Methods in general, you have to fit one last missing piece in the Ruby object model.

Singleton Classes

Where you place the final piece in the object model puzzle.

Singleton classes are the UFOs of the Ruby world: even if you never see one in person, you can find scattered hints of their existence all over the place. Let's start our investigation into this difficult subject by collecting some evidence. (Be aware that the next few pages contain advanced material that might take a while for you to digest. If you want, you can skip straight to *Method Wrappers*, on page 131, on your first read through and come back to this section later.)

The Mystery of Singleton Methods

In *Method Lookup*, on page 28, you learned how Ruby finds methods by going *right* into the receiver's class and then *up* the class hierarchy. For example:

```
class MyClass
  def my_method; end
end
```